

19. sim 用車体状態 publisher

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/solar_state_node.py
種別	Python
区分	runtime node
役割	sim モードで planner 指令速度から速度、距離、電池、PV、altitude を模擬 publish する。
起動文脈	simulation launch における擬似車両。

このファイルを読む理由

sim モードで planner 指令速度から速度、距離、電池、PV、altitude を模擬 publish する。

- SolarCarModel を直接生成する。
- /planner/speed_cmd を受けて /vehicle/* を出す。

呼び出し関係

どこから呼ばれるか

- launch/solarcar_sim.launch.py

次に読むと理解が進むファイル

- mpc_solarcar/model.py
- mpc_solarcar/mpc_node.py

同一リポジトリ内の主要依存

- mpc_solarcar/model.py
- mpc_solarcar/path_utils.py
- mpc_solarcar/route_utils.py
- mpc_solarcar/schedule_utils.py
- mpc_solarcar/signal_utils.py
- mpc_solarcar/solar_profile.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from .solar_profile import get_path, get_section, load_profile, require_csv_data_rows

class SolarStateNode(Node):
    def __init__(self) -> None:
        super().__init__("solar_state_node")
        self.declare_parameter("profile_yaml", "")
        self.declare_parameter("forecast_csv", "inputs/forecast_10min.csv")
        self.declare_parameter("route_profile_csv", "")
        self.declare_parameter("drive_schedule_yaml", "")
        self.declare_parameter("drive_eff_map", "")
        self.declare_parameter("regen_eff_map", "")
        self.declare_parameter("rint_map", "")
        self.declare_parameter("panel_eff_map", "")
        self.declare_parameter("mppt_eff_map", "")
        self.declare_parameter("drive_map_eco", "")
        self.declare_parameter("drive_map_power", "")
        self.declare_parameter("regen_map_eco", "")
        self.declare_parameter("regen_map_power", "")
        self.declare_parameter("ocv_soc_map", "")
        self.declare_parameter("params_yaml", "")
        self.declare_parameter("forecast_time_mode", "auto")
```

主要構造の読み取り

主要クラスは SolarStateNode。主要関数は main。ROS パラメータ宣言は 26 件。ROS I/O は publisher 8 件、subscription 1 件。

主要クラス・関数

行	種別	名前	補足
22	class	SolarStateNode	bases: Node
23	function	__init__	args: self
180	function	_on_speed_cmd	args: self, msg
183	function	_forecast_row	args: self, now_utc
191	function	_route_value	args: self, field, default
199	function	_step	args: self
265	function	main	

ファイルを上から読んだときの定義順

行	内容
22	クラス SolarStateNode を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や <code>forward reference</code> を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>importmath</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での使用位置は少ないか、間接利用である。
4	<code>importtime</code>	<code>wall/monotonic time</code> に基づく周期制御や <code>freshness</code> 判定を行うため。このファイル内での使用位置は少ないか、間接利用である。
5	<code>fromdatetimeimportdatetime, timedelta,timzone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L118, L122, L183, L209。
7	<code>importnumpyasnp</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L185, L186, L188, L240。
8	<code>importpandasaspd</code>	CSV や時刻列の読み込み・整形・再サンプリングに使うため。このファイル内での主な使用位置は L99, L105, L107。
9	<code>importrcipy</code>	ROS 2 Python ノードとして起動・ <code>spin</code> するため。このファイル内での主な使用位置は L59, L60, L61, L62, L63, L64, L65, L66, ...。
10	<code>fromrcipy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L22。
12	<code>fromstd_msgs. msgimportFloat32</code>	<code>Float32</code> や <code>String</code> など軽量メッセージで <code>planner/vehicle</code> 値を <code>publish</code> するため。このファイル内での主な使用位置は L168, L169, L170, L171, L172, L173, L174, L175, ...。
14	<code>from.modelimportParams, SolarCarModel</code>	車体物理・電気モデル本体から <code>Params</code> 、 <code>SolarCarModel</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/model.py</code> 。このファイル内での主な使用位置は L128, L132。
15	<code>from.path_ utilsimportresolve_path</code>	ROS share / 相対パス解決から <code>resolve_path</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/path_utils.py</code> 。このファイル内での主な使用位置は L87, L95, L103, L129, L130, L131, L133, L134, ...。
16	<code>from.route_ utilsimportinterpolate_ profile</code>	<code>route_utils.py</code> から <code>interpolate_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/route_utils.py</code> 。このファイル内での主な使用位置は L195。
17	<code>from.schedule_ utilsimportDriveSchedule</code>	<code>schedule_utils.py</code> から <code>DriveSchedule</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/schedule_utils.py</code> 。このファイル内での主な使用位置は L103。
18	<code>from.signal_ utilsimportSmoothRateLimiter</code>	<code>signal_utils.py</code> から <code>SmoothRateLimiter</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/signal_utils.py</code> 。このファイル内での主な使用位置は L158。

```
19 from.solar_
    profileimportget_path,
    get_section,load_profile,
    require_csv_data_rows
```

profile YAML 読込と検証から `get_path`, `get_section`, `load_profile`, `require_csv_data_rows` を読み込み、このファイルの内部処理を分担させるため。実体ファイルは `mpc_solarcar/solar_profile.py`。このファイル内の主な使用位置は L54, L55, L56, L59, L60, L61, L62, L63, ...。

関数・クラスを上から順に解説

L22 クラス `SolarStateNode`

- 定義: `SolarStateNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `Parameter`, `Params`, `SmoothRateLimiter`, `SolarCarModel`, `__init__`, `_forecast_row`, `_route_value`, `abs`, `any`, `astimezone`, `clip`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. 関数 `__init__` を定義する。
2. 関数 `_on_speed_cmd` を定義する。
3. 関数 `_forecast_row` を定義する。
4. 関数 `_route_value` を定義する。
5. 関数 `_step` を定義する。

L265 関数 `main`

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `SolarStateNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. `rclpy.init(...)` を実行する。
2. `node` に `SolarStateNode()` の結果を代入する。
3. `rclpy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rclpy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
25	<code>profile_yaml</code>	
26	<code>forecast_csv</code>	<code>inputs/forecast_10min.csv</code>

27	route_profile_csv	
28	drive_schedule_yaml	
29	drive_eff_map	
30	regen_eff_map	
31	rint_map	
32	panel_eff_map	
33	mppt_eff_map	
34	drive_map_eco	
35	drive_map_power	
36	regen_map_eco	
37	regen_map_power	
38	ocv_soc_map	
39	params_yaml	
40	forecast_time_mode	auto
41	forecast_time_tz	UTC
42	forecast_start_time_utc	
43	publish_rate_hz	2.0
44	init_speed_kmh	45.0
45	soc0	0.95
46	Tb0	30.0
47	s0_km	0.0
48	filter_tau_sec	1.0
49	accel_limit_kmhps	1.2
50	decel_limit_kmhps	3.5

ROS topic I/O

行	種別	topic	callback / 補足
168	publisher	/vehicle/speed_kmh	publisher
169	publisher	/vehicle/s_km	publisher
170	publisher	/vehicle/batt_soc	publisher
171	publisher	/vehicle/batt_temp_c	publisher
172	publisher	/vehicle/batt_current_a	publisher
173	publisher	/vehicle/batt_voltage_v	publisher
174	publisher	/vehicle/solar_power_w	publisher
175	publisher	/vehicle/altitude_m	publisher
176	subscription	/planner/speed_cmd	self._on_speed_cmd

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_state_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。